

# Distributed Observability Demo Application

## Overview

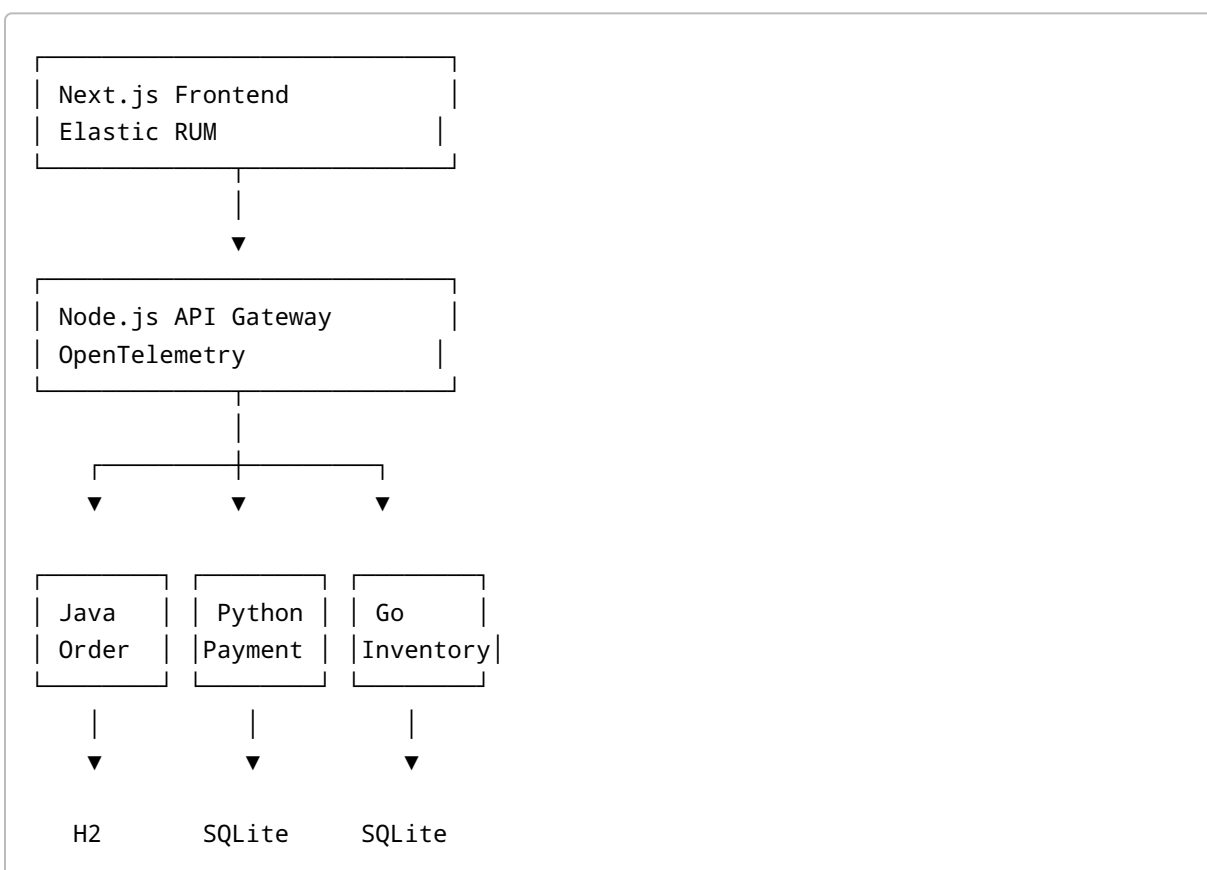
This project demonstrates Full Stack Observability using:

- Elastic Observability
- OpenTelemetry
- Elastic RUM
- Distributed Tracing
- Service Maps
- Application Metrics
- Error Monitoring
- Log Correlation

The goal is to provide a realistic microservices environment for demonstrations, workshops, and training.

---

## Architecture



---

# Business Flows

The application implements only two business flows to keep demonstrations simple and easy to follow.

---

## Flow 1: Purchase Product

### User Journey

```
graph TD; A[Browse Products] --> B[View Product]; B --> C[Add To Cart]; C --> D[Checkout]; D --> E[Payment Success]; E --> F[Order Created];
```

A vertical sequence of steps in a user journey, connected by downward arrows. The steps are: Browse Products, View Product, Add To Cart, Checkout, Payment Success, and Order Created.

### Service Flow

```
graph TD; A[Frontend] --> B[Gateway]; B --> C[Inventory Service]; B --> D[Payment Service]; B --> E[Order Service];
```

A diagram showing the service flow. It starts with 'Frontend' at the top, which connects via a vertical line with a downward arrow to 'Gateway'. From 'Gateway', three separate vertical lines branch out to the right, each ending in a horizontal line that connects to a service name: 'Inventory Service', 'Payment Service', and 'Order Service'.

### Observability Demonstration

- Browser RUM
  - Distributed Tracing
  - Service Map
  - Database Spans
  - Metrics Correlation
-

## Flow 2: Refund Order

### User Journey

```
graph TD; A[Search Order] --> B[View Order]; B --> C[Refund Order]; C --> D[Refund Payment]; D --> E[Restock Inventory];
```

### Service Flow

```
graph TD; A[Frontend] --> B[Gateway]; B --> C[Order Service]; B --> D[Payment Service]; B --> E[Inventory Service];
```

### Observability Demonstration

- Alternative Trace Path
- Error Monitoring
- Log Correlation
- Slow Service Analysis

## Technology Stack

| Component     | Technology      |
|---------------|-----------------|
| Frontend      | Next.js         |
| Gateway       | Node.js Express |
| Order Service | Spring Boot     |

| Component           | Technology            |
|---------------------|-----------------------|
| Payment Service     | FastAPI               |
| Inventory Service   | Go Gin                |
| Order Database      | H2                    |
| Payment Database    | SQLite                |
| Inventory Database  | SQLite                |
| Observability       | OpenTelemetry         |
| Frontend Monitoring | Elastic RUM           |
| Backend Monitoring  | Elastic Observability |

---

## Service Details

### Frontend

#### Folder

frontend/

#### Responsibilities

- Product Catalog
- Product Detail
- Checkout
- Order History
- Refund Requests

#### Instrumentation

Elastic RUM Agent

Captures:

- Page Load
- Core Web Vitals
- User Actions
- API Requests
- Browser Errors

## API Gateway

### Folder

gateway/

### Technology

Node.js + Express

### Responsibilities

- API Entry Point
- Request Routing
- Correlation Context Propagation

### Instrumentation

OpenTelemetry Node SDK

---

## Order Service

### Folder

services/order-service/

### Technology

Spring Boot

### Database

H2 Embedded Database

### Responsibilities

- Create Order
- Retrieve Order
- Refund Order

## Example Endpoints

```
POST /orders  
  
GET /orders/{id}  
  
POST /orders/{id}/refund
```

## Observability Goals

- HTTP Tracing
  - JDBC Tracing
  - Exception Monitoring
- 

## Payment Service

### Folder

```
services/payment-service/
```

### Technology

FastAPI

### Database

SQLite

### Responsibilities

- Process Payment
- Refund Payment
- Payment History

## Example Endpoints

```
POST /payments  
  
POST /payments/refund  
  
GET /payments/{id}
```

## Observability Goals

- HTTP Tracing
  - SQLite Queries
  - Error Demonstration
- 

## Inventory Service

### Folder

```
services/inventory-service/
```

### Technology

Go Gin

### Database

SQLite

### Responsibilities

- Check Inventory
- Reserve Inventory
- Restock Inventory

### Example Endpoints

```
GET /inventory/{sku}

POST /inventory/reserve

POST /inventory/restock
```

## Observability Goals

- HTTP Tracing
  - Database Spans
  - Latency Analysis
-

# Repository Structure

distributed-observability-demo/

```
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   └── Dockerfile
├── gateway/
│   ├── src/
│   ├── package.json
│   └── Dockerfile
├── services/
│   ├── order-service/
│   │   ├── src/
│   │   ├── pom.xml
│   │   └── Dockerfile
│   ├── payment-service/
│   │   ├── app/
│   │   ├── requirements.txt
│   │   └── Dockerfile
│   └── inventory-service/
│       ├── cmd/
│       ├── go.mod
│       └── Dockerfile
├── observability/
│   ├── otel-collector/
│   │   └── config.yaml
│   ├── elastic/
│   │   └── onboarding.md
│   └── rum/
│       └── config.js
├── deployment/
│   ├── kubernetes/
│   │   └── frontend.yaml
```



```
| | | | gateway.yaml
| | | | order-service.yaml
| | | | payment-service.yaml
| | | | inventory-service.yaml
| | | |
| | | | openshift/
| | | | | frontend.yaml
| | | | | gateway.yaml
| | | | | order-service.yaml
| | | | | payment-service.yaml
| | | | | inventory-service.yaml
| | | |
| | | | README.md
```

# OpenTelemetry Strategy

## Frontend

Elastic RUM

```
Browser
  ↓
Elastic Cloud
```

## Backend

OpenTelemetry

```
Application
  ↓
OTLP
  ↓
OpenTelemetry Collector
  ↓
Elastic Cloud
```

# Demonstration Scenarios

## Scenario 1

Successful Purchase

Demonstrates:

- Distributed Tracing
  - Service Map
  - Database Spans
  - Browser Monitoring
- 

## Scenario 2

Payment Failure

Demonstrates:

- Error Monitoring
  - Stack Traces
  - Log Correlation
- 

## Scenario 3

Slow Inventory Service

Demonstrates:

- Latency Analysis
  - Bottleneck Detection
  - Trace Waterfall Analysis
- 

# Training Objectives

Participants will learn:

- OpenTelemetry Fundamentals
- Elastic Observability
- Distributed Tracing

- RUM Monitoring
  - Service Maps
  - Error Tracking
  - Log Correlation
  - Root Cause Analysis
  - Observability Best Practices
- 

## Design Principles

- Small enough to run in a lab environment
- Multiple programming languages
- Multiple database technologies
- Realistic business flows
- Cloud-native architecture
- OpenTelemetry-first instrumentation
- Elastic-native visualization